kauno kolegija | Technologijų fakultetas

.NET frontend technologijos

Razor Pages, Blazor Server ir Blazor WebAssembly su tuo pačiu SimpleStudent API

Grupė PS-5

2026 m. ruduo

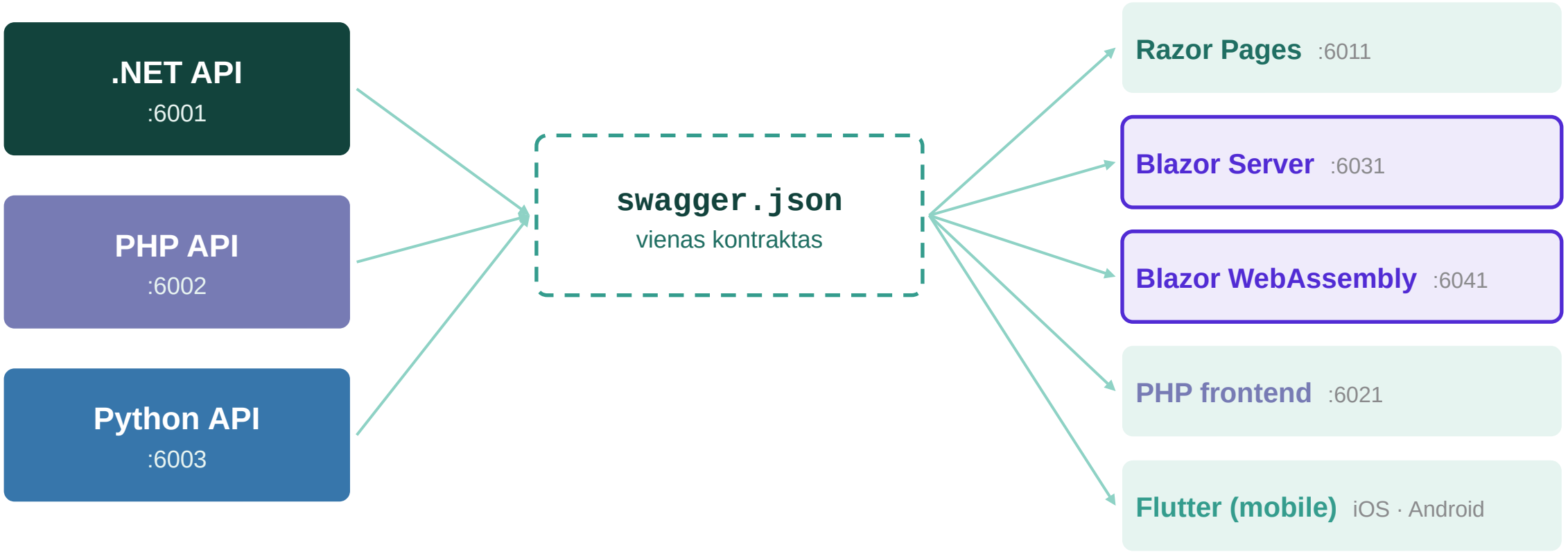
Paskaita + praktika

Dėstytojas: Tadas Bulis

KUR TAI TELPA

Vienas API kontraktas, daug klientų

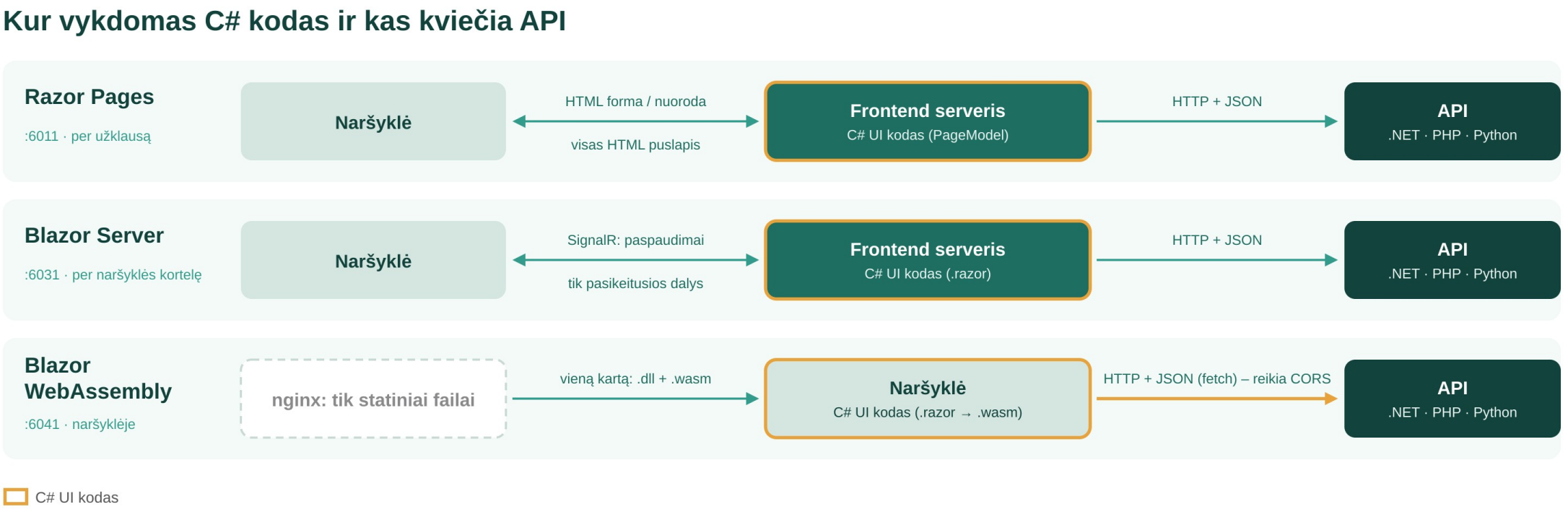
Trys API realizacijos ir penki klientai, visi pagal tą patį swagger.json



Violetiniu rėmeliu pažymėti nauji Blazor klientai

Kur vykdomas C# kodas ir kas kviečia API

Tas pats API, tas pats sugeneruotas klientas, skirtinga vieta UI kodui



Razor Pages: puslapis per užklausą

SimpleStudentFrontend · :6011 · .cshtml + PageModel

1. **GET /Students** – naršyklė prašo puslapio
2. **OnGetAsync()** serveryje kviečia API per IStudentApiClient
3. **Visas HTML** sugeneruojamas ir išsiunčiamas; PageModel sunaikinamas
4. **Veiksmas = nauja užklausa**: POST → RedirectToPage() → GET

- Paprasčiausias ir patikimiausias modelis: veikia ir be WebSocket, puikiai tinka SEO
- Interaktyvumui be perkrovimų reikia JavaScript

Blazor Server: komponentai gyvena serveryje

SimpleStudentBlazorServer · :6031 · .razor komponentai, render mode Interactive Server

1. **blazor.web.js** atidaro SignalR (WebSocket) jungtį
2. **Circuit**: serveryje kiekvienai kortelei laikoma komponentų būsena
3. **Paspaudimas** keliauja per SignalR; serveris vykdo C# metodą
4. **Tik pakeitimai** grįžta į naršyklę; API kviečia serveris

- Interaktyvu be JavaScript, greitas pirmas įkėlimas, kodas ir raktai lieka serveryje
- Kiekvienas vartotojas = atmintis serveryje; nutrūkus ryšiui UI sustoja („reconnect“)

Blazor WebAssembly: programa naršyklėje

SimpleStudentBlazorWasm · :6041 · tie patys .razor komponentai + nginx

1

nginx atiduoda index.html, .wasm ir .dll – serveryje .NET nėra

→

2

.NET runtime paleidžiamas naršyklėje (WebAssemblyHostBuilder)

→

3

Komponentai vykdomi naršyklėje – paspaudimams serverio nereikia

→

4

API kviečia naršyklė per fetch() → reikia CORS ir viešo adreso

- Statinis talpinimas (CDN), veikia nutrūkus ryšiui, apkrova perkeliama klientui
- Didesnis pirmas atsisiuntimas; visi .dll ir wwwroot/appsettings.json yra vieši

PALYGINIMAS

Trys technologijos vienoje lentelėje

	Razor Pages	Blazor Server	Blazor WebAssembly
Kur vykdomas UI C# kodas	Serveryje, per užklausą	Serveryje, per kortelę	Naršyklėje
Puslapio atnaujinimas	Visas puslapis	Tik pasikeitusios dalys	Tik pasikeitusios dalys
Kas kviečia API	Serveris	Serveris	Naršyklė
CORS API pusėje	Nereikia	Nereikia	Reikia
Pirmas įkėlimas	Greitas	Greitas	Lėtesnis (keli MB)
Serverio apkrova	Tik užklausos metu	Atmintis kiekvienam vartotojui	Minimali (statiniai failai)
Nutrūkus tinklui	Veikia kitas paspaudimas	UI sustoja, „reconnect“	UI veikia, API – ne
Slaptos reikšmės	Gali būti serveryje	Gali būti serveryje	Negalima – viskas vieša
Talpinimas	ASP.NET Core	ASP.NET Core	nginx / bet koks statinis
Mūsų projektas	:6011	:6031	:6041

TAS PATS UŽDAVINYS

Studentų sąrašo įkėlimas

Razor Pages vs Blazor (Server ir WASM kodas identiškas)

Razor Pages — Pages/Students.cshtml.cs

```
public class StudentsModel : PageModel
{
  public List<Student> Students { get; set; }

  public async Task OnGetAsync()
  {
    // GET /api/students
    Students = (await _api.GetStudentsAsync())
                    .ToList();
  }
}
```

Blazor — Pages/Students.razor

```
@page "/students"
@inject ApiConnection Api

<input @bind="_search"
       @bind:event="oninput" />
@foreach (var s in Filtered) { <tr>…</tr> }

@code {
  List<Student> _students = new();
  protected override async Task
      OnInitializedAsync() => _students =
      (await Api.Client.GetStudentsAsync()).ToList();
}
```

Razor Pages: duomenys įkeliami užklausos metu, šablonas atvaizduoja kartą. **Blazor:** pasikeitus laukui, Blazor pats perpiešia tą HTML dalį.

TAS PATS UŽDAVINYS

Studento trynimas

Forma ir peradresavimas vs vienas C# metodas

Razor Pages — Students.cshtml(.cs)

```
<form method="post"
      asp-page-handler="Delete">
  <input type="hidden" name="id"
         value="@s.Id" />
  <button type="submit">Delete</button>
</form>

public async Task<IActionResult>
    OnPostDeleteAsync(int id)
{
  await _api.DeleteStudentAsync(id);
  return RedirectToPage(); // visas puslapis
}
```

Blazor — Pages/Students.razor

```
<button @onclick="() => DeleteAsync(s.Id)">
  Yes
</button>

@code {
  async Task DeleteAsync(int id)
  {
    await Api.Client.DeleteStudentAsync(id);
    _students.RemoveAll(s => s.Id == id);
    // dingsta tik viena eilutė
  }
}
```

Razor Pages: POST → 302 → GET, perkraunamas visas puslapis. **Blazor:** vienas metodas, naršyklėje pasikeičia viena eilutė.

Program.cs: Blazor Server vs WebAssembly

Puslapių komponentai tie patys, skiriasi tik tai, kur jie paleidžiami

SimpleStudentBlazorServer/Program.cs

```
// vykdoma SERVERYJE
builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();
builder.Services.AddScoped<ApiConnection>();
...
app.MapRazorComponents<App>()
   .AddInteractiveServerRenderMode();

// Components/App.razor
<Routes @rendermode="new
  InteractiveServerRenderMode(prerender: false)"/>
```

SimpleStudentBlazorWasm/Program.cs

```
// vykdoma NARŠYKLĖJE
var builder = WebAssemblyHostBuilder
                  .CreateDefault(args);
builder.RootComponents.Add<App>("#app");
builder.Services.AddScoped<ApiConnection>();
await builder.Build().RunAsync();

// Pages/*.razor, Layout/*, Shared/*
// – TIE PATYS failai abiejuose projektuose
```

Razor Pages registruoja AddRazorPages() + MapRazorPages(). **prerender: false** – kad API nebūtų kviečiamas du kartus.

Validacijos taisyklės ateina iš kontrakto

Shared/StudentForm.razor – viena forma kūrimui ir redagavimui



Blazor — Shared/StudentForm.razor

```
<EditForm EditContext="_editContext"
          OnValidSubmit="SubmitAsync" novalidate>
  <DataAnnotationsValidator />
  <InputText @bind-Value="Model.FirstName" />
  <ValidationMessage For="() => Model.FirstName"/>
</EditForm>
```

Spąstai: type="email"

Naršyklės HTML5 validacija tyliai blokuoja formą su reikšme „x“: nesimato nei Blazor, nei API klaidų.

Sprendimas: novalidate ant EditForm. Šią klaidą radome testuodami pavyzdį.

KAS KVIEČIA API

Serverio adresas ar naršyklės adresas?

Svarbiausias praktinis skirtumas tarp Blazor Server ir WebAssembly

Razor Pages ir Blazor Server

API kviečia serveris Docker tinkle (simplestudentapi:8080). CORS nereikia, raktai gali likti serveryje.

Blazor WebAssembly

API kviečia naršyklė adresu, kurį ji mato (localhost:6001). Reikia CORS, visa wwwroot yra vieša.

docker-compose.yml (Blazor Server)

```
Backends__0__BaseUrl:
  http://simplestudentapi:8080
```

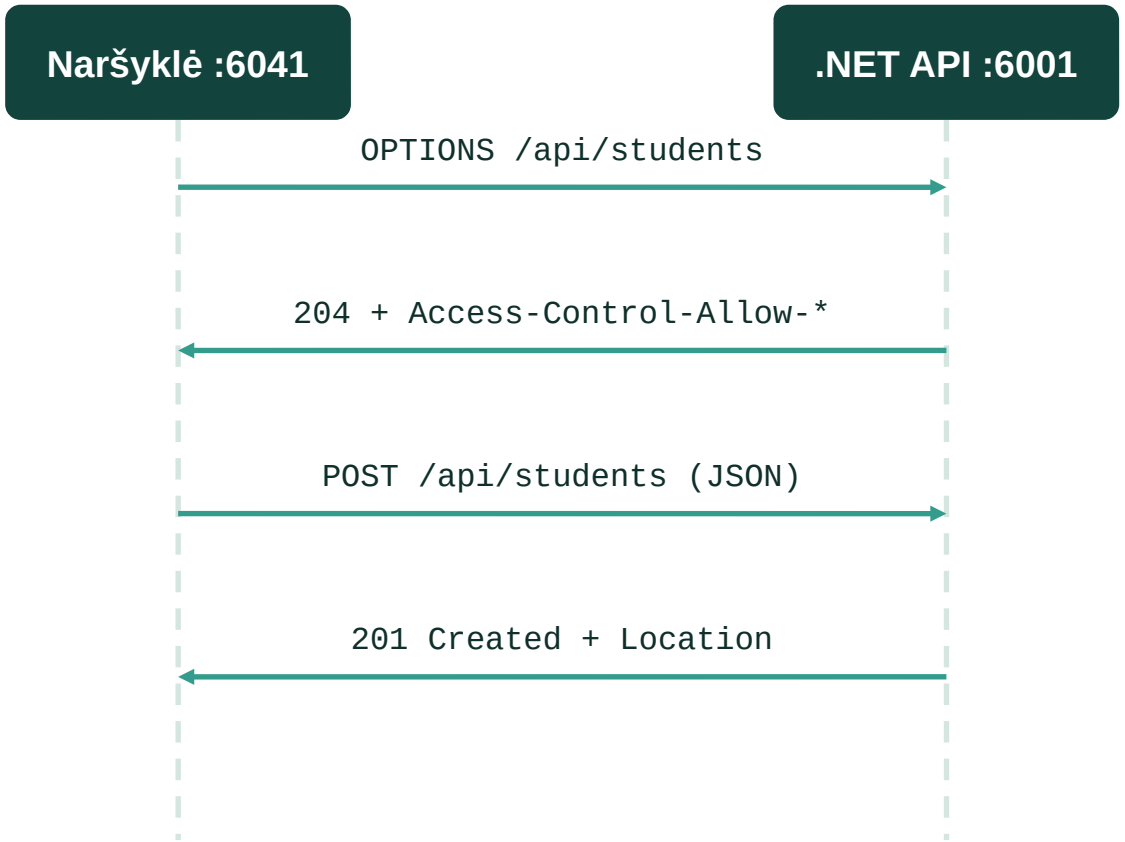
wwwroot/appsettings.json (WebAssembly)

```
{ "Name": ".NET",
  "Url": "http://localhost:6001" }
```

F12 → Network: Razor Pages ir Blazor Server lange užklausų į :6001 nematysite, o WebAssembly lange matysite kiekvieną.

Kodėl CORS reikia tik WebAssembly

Naršyklė klausia API leidimo prieš siųsdama JSON iš kito adreso



.NET API — SimpleStudentApi/Program.cs

```
builder.Services.AddCors(options =>
  options.AddDefaultPolicy(policy => policy
    .AllowAnyOrigin()
    .AllowAnyHeader()
    .AllowAnyMethod()
    .WithExposedHeaders("Location")));
...
app.UseCors();
```

Kodėl tik WebAssembly?

CORS yra naršyklės taisyklė, serveriai jos nevykdo. PHP ir Python API CORS jau turėjo, .NET API papildėme.

PASIRINKIMAS

Kada ką rinktis

Sprendimą lemia vartotojai, tinklas ir talpinimas, o ne API

Razor Pages

Rinkitės, kai…

- turinio svetainė, formos
- svarbus SEO
- reikia paprasčiausio sprendimo

Atsargiai, jei…

- reikia daug interaktyvumo be perkrovimų

Blazor Server

Rinkitės, kai…

- vidinė verslo programa
- daug interaktyvumo
- API ir DB privačiame tinkle

Atsargiai, jei…

- daug vartotojų vienu metu
- nestabilus tinklas

Blazor WASM

Rinkitės, kai…

- statinis talpinimas, CDN
- veikimas be ryšio (PWA)
- apkrova perkeliama klientui

Atsargiai, jei…

- didelis pirmas atsisiuntimas
- API turi būti viešas + CORS

Ką sutiksite praktikoje

Visas keturias matėme kurdami pavyzdžius

BLAZOR SERVER

API kviečiamas du kartus

Prerender sugeneruoja puslapį du kartus. Pas mus: InteractiveServerRenderMode(prerender: false).

BLAZOR SERVER

„Attempting to reconnect…“

Nutrūko SignalR jungtis. Puslapis prisijungs pats, bet circuit būsena (pvz., API pasirinkimas) prarandama.

BLAZOR WASM

„TypeError: Failed to fetch“

API neveikia arba neleidžia CORS. F12 → Console parodo, kurios antraštės trūksta.

BLAZOR WASM

simplestudentapi:8080 neveikia

Šis vardas egzistuoja tik Docker tinkle. Naršyklei reikia localhost:6001.

DEMONSTRACIJA

Paleidimas ir demonstracija

Iš aplanko student-docker-lab-plus-mssql

Terminalas

```
docker-compose -f docker-compose.infra.yml up -d
docker-compose up --build
```

Demonstracijos eiga

1. Atidarykite :6031 ir :6041 greta
1. Perjunkite API į Python – puslapis neperkraunamas
1. F12 → Network ir ištrinkite studentą abiejuose
1. docker stop simplestudentblazorserver – stebėkite „reconnect“

:6011	Razor Pages
:6031	Blazor Server
:6041	Blazor WebAssembly
:6021	PHP frontend'as
:6001 · 6002 · 6003	.NET · PHP · Python API

Užduotys studentams

Išsamiai aprašytos dokumente DotNet-Frontend-Palyginimas_LT

1. F12 → Network: suskaičiuokite užklausas trinant studentą **visuose trijuose** frontend'uose

2. Sustabdykite Blazor Server, vėliau WASM konteinerį. **Kuo skiriasi** elgesys?

3. Pašalinkite `app.UseCors()` iš .NET API. Kodėl veikia tik serverio frontend'ai?

4. Blazor sąraše pridėkite **rūšiavimą pagal pavardę** be naujos API užklausos

5. Razor Pages frontend'e sukurkite **Faculty** puslapį ir palyginkite kodo kiekį

6. Su PHP API išsaugokite el. paštą „x“. **Iš kur** atėjo klaidos pranešimas?

Tas pats API, trys būdai jį rodyti

Skiriasi tik tai, kur vykdomas C# kodas ir kas siunčia HTTP užklausą

Razor Pages
serveris, visas puslapis per užklausą

Blazor Server
serveris, tik pakeitimai per SignalR

Blazor WebAssembly
naršyklė, API kviečiamas tiesiogiai + CORS

Dėstytojas: Tadas Bulis